

Основы функционального программирования

Практическая работа №5

Ленивые вычисления и бесконечные структуры

Цель: Освоить использование ленивости для создания эффективных структур данных и потоков.

Теоретическая справка

- Ленивость — вычисления откладываются до момента использования.
- Бесконечные списки: `ones = 1 : ones`, `nats = 1 : map (+1) nats`
- Мемоизация — повторное использование уже вычисленных значений благодаря ленивости.
- `take`, `drop`, `!!` — инструменты работы с потоками.
- `seq` и `deepseq` — принудительное вычисление при необходимости.

Пример решения

Задача: Реализовать бесконечный список чисел Фибоначчи и функцию поиска индекса числа.

```
-- Бесконечный список Фибоначчи через ленивую рекурсию
fibs :: [Integer]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)

-- Поиск индекса числа в последовательности (возвращает Nothing,
если нет)
findFibIndex :: Integer -> Maybe Int
findFibIndex n = go 0 fibs
  where
    go _ []      = Nothing
    go i (x:xs)
      | x == n    = Just i
      | x > n     = Nothing    -- оптимизация: числа растут
                               -- монотонно
      | otherwise = go (i + 1) xs

-- Сумма чётных чисел Фибоначчи до заданного предела
```

```
sumEvenFibsUnder :: Integer -> Integer
sumEvenFibsUnder limit =
    sum . takeWhile (< limit) . filter even $ fibs
```

Тестирование в GHCi

```
-- > take 10 fibs
-- [0,1,1,2,3,5,8,13,21,34]
-- > findFibIndex 34
-- Just 9
-- > sumEvenFibsUnder 100
-- 44 -- 2 + 8 + 34
```

Почему это эффективно?

- `fibs` вычисляется один раз и кэшируется благодаря ленивости.
- `takeWhile` останавливает вычисление, как только условие нарушено.
- Нет избыточных вычислений при повторных обращениях к `fibs !! n`.

Ход работы

1. Создайте файл `Pr4_V<N>.hs`.
2. Определите бесконечную структуру (список, дерево) через самореференцию.
3. Реализуйте функции доступа через `take`, `drop`, `!!`.
4. Добавьте оптимизации (например, остановку при превышении предела).
5. Протестируйте производительность на больших значениях (например, `fibs !! 10000`).

Требования к отчёту

- Титульный лист (ФИО, группа, работа №1, вариант)
- Условие задачи
- Листинг кода с комментариями
- Объяснение механизма ленивости на примере вашей структуры
- Замеры времени выполнения (через `:set +s` в GHCi) для разных входов
- Сравнение с энергичной реализацией (теоретическое)
- Формат PDF

Варианты заданий

Вариант 1. Бесконечный список простых чисел (Решето Эратосфена)

Компонент	Описание
Задача	Реализация бесконечного списка простых чисел через ленивую версию решета Эратосфена
<code>primes :: [Integer]</code>	Бесконечный список всех простых чисел в порядке возрастания
<code>nthPrime :: Int -> Integer</code>	Возвращает <code>n</code> -е простое число (нумерация с 0)
<code>primesInRange :: Integer -> Integer -> [Integer]</code>	Все простые числа в диапазоне <code>[a, b]</code> (включительно)
Граничные случаи	• <code>nthPrime</code> с отрицательным индексом — не определено (можно вернуть <code>error</code>) • <code>primesInRange</code> с <code>a > b</code> → <code>[]</code> • <code>primesInRange</code> с отрицательным <code>a</code> → начинать с 2
Эффективность	• Использовать ленивую реализацию решета (не решето на массиве)

Вариант 2. Последовательность Фибоначчи

Компонент	Описание
Задача	Бесконечный список чисел Фибоначчи с мемоизацией через ленивость
<code>fibs :: [Integer]</code>	Бесконечный список Фибоначчи: <code>0, 1, 1, 2, 3, 5, 8, ...</code>
<code>fib :: Int -> Integer</code>	Возвращает <code>n</code> -е число Фибоначчи (нумерация с 0)
<code>sumEvenFibsUnder :: Integer -> Integer</code>	Сумма всех чётных чисел Фибоначчи, меньших заданного предела
Граничные случаи	• <code>fib</code> с отрицательным индексом — не определено • <code>sumEvenFibsUnder</code> с отрицательным пределом → <code>0</code>

Эффективность	• Реализация через <code>zipWith</code>: <code>fibs = 0 : 1 : zipWith (+) fibs (tail fibs)</code> • Мемоизация: повторные обращения к <code>fibs !! n</code> не пересчитываются
---------------	--

Вариант 3. Поток случайных чисел

Компонент	Описание
Задача	Генерация бесконечного потока псевдослучайных чисел без IO
<code>randomStream :: Int -> [Int]</code>	Бесконечный поток целых чисел от 0 до 99, используя генератор с начальным значением (семенем)
<code>linearCongruential :: Int -> Int</code>	Генератор псевдослучайных чисел по формуле: <code>next = (a * seed + c) mod m</code> Параметры: <code>a = 1103515245</code> , <code>c = 12345</code> , <code>m = 2³¹</code>
<code>takeUnique :: Eq a => Int -> [a] -> [a]</code>	Берёт из потока первые <code>n</code> уникальных элементов
Граничные случаи	• <code>takeUnique 0 xs</code> → <code>[]</code> • <code>takeUnique n []</code> → <code>[]</code> (если недостаточно уникальных элементов)
Эффективность	• Бесконечный поток определяется рекурсивно: <code>randoms seed = val : randoms next</code> • Для получения чисел в диапазоне использовать <code>mod 100</code>

Вариант 4. Мемоизированный факториал и числа Каталана

Компонент	Описание
Задача	Реализация мемоизированных рекурсивных функций через бесконечные списки
<code>fact :: Int -> Integer</code>	Факториал с мемоизацией. Использовать бесконечный список <code>facts = map factorial [0..]</code>
<code>factorial :: Int -> Integer</code>	Вспомогательная функция для построения списка (рекурсивная, использует <code>facts</code>)
<code>catalan :: Int -> Integer</code>	Числа Каталана: $C_0 = 1, C_{n+1} = \sum_{i=0..n} C_i \cdot C_{n-i}$ Реализовать через мемоизированный

	бесконечный список
Граничные случаи	• Отрицательные аргументы — не определены (вернуть <code>error</code>) • Большие значения (<code>fact 1000</code>) должны вычисляться быстро благодаря мемоизации
Эффективность	• Без мемоизации <code>catalan 20</code> будет работать очень долго (экспоненциальная сложность) • С мемоизацией через <code>catalans = map calcCatalan [0..]</code> — линейная сложность

Вариант 5. Бесконечное дерево путей в решёточной сетке

Компонент	Описание
Задача	Генерация всех путей из точки $(0,0)$ в точку (x,y) на решёточной сетке (только вправо и вверх)
Тип <code>Path</code>	<code>type Path = [(Int, Int)]</code> — список координат от $(0,0)$ до конечной точки
<code>pathsTo :: (Int, Int) -> [[(Int, Int)]]</code>	Все возможные пути из $(0,0)$ в (x,y) . Каждый шаг — либо вправо $(x+1,y)$, либо вверх $(x,y+1)$
<code>countPaths :: (Int, Int) -> Integer</code>	Количество путей (без генерации самих путей). Формула: $C(x+y, x)$
Граничные случаи	• <code>pathsTo (0,0) → [[(0,0)]]</code> (один путь — пустое перемещение) • Отрицательные координаты — не определены
Эффективность	• Ленивая генерация: не все пути вычисляются сразу, только при обращении через <code>take</code> • Для больших координат использовать <code>countPaths</code> вместо генерации всех путей

Вариант 6. Поток всех рациональных чисел (Перечисление Кантора)

Компонент	Описание
Задача	Диагональное перечисление всех положительных рациональных чисел без дубликатов

<code>rational :: [(Integer, Integer)]</code>	Бесконечный список пар (числитель, знаменатель), представляющих все положительные рациональные числа в канонической форме (несократимые дроби)
<code>gcd' :: Integer -> Integer -> Integer</code>	Наибольший общий делитель (для сокращения дробей)
<code>isReduced :: (Integer, Integer) -> Bool</code>	Проверка, что дробь несократима (<code>gcd = 1</code>)
Граничные случаи	- Знаменатель всегда > 0 - Исключаются дубликаты: $(1, 2)$ включено, $(2, 4)$ исключено
Эффективность	- Диагональное перечисление: сначала сумма числитель+знаменатель = 2, потом 3, потом 4 и т.д. - Фильтрация через <code>filter isReduced</code>

Вариант 7. Ленивая матрица Паскаля

Компонент	Описание
Задача	Бесконечная матрица треугольника Паскаля, доступная по координатам
<code>pascal :: [[Integer]]</code>	Бесконечный список списков — треугольник Паскаля
<code>pascalRow :: Int -> [Integer]</code>	n -я строка треугольника Паскаля (нумерация с 0)
<code>pascalAt :: Int -> Int -> Integer</code>	Элемент на позиции (строка, столбец) (биномиальный коэффициент $C(n, k)$)
<code>binomial :: Integer -> Integer -> Integer</code>	Биномиальный коэффициент $C(n, k)$ через <code>pascalAt</code>
Граничные случаи	<code>pascalAt n k</code> при $k > n \rightarrow 0$ - Отрицательные индексы — не определены
Эффективность	- Рекурсивное определение: <code>pascal = repeat 1 : zipWith (\xs ys -> zipWith (+) xs (0:ys)) pascal (tail pascal)</code> - Мемоизация: повторные обращения к <code>pascalAt</code> не пересчитываются

Вариант 8. Генерация всех перестановок (лексикографический поток)

Компонент	Описание
Задача	Бесконечный поток всех возможных строк из заданного алфавита в лексикографическом порядке
<code>lexStream :: [a] -> [[a]]</code>	Поток всех строк из алфавита, упорядоченных сначала по длине, затем лексикографически
<code>stringsOfLength :: Int -> [a] -> [[a]]</code>	Все строки заданной длины из алфавита
<code>nextString :: Ord a => [a] -> [a] -> Maybe [a]</code>	Следующая строка в лексикографическом порядке (если существует)
Граничные случаи	• Пустой алфавит → <code>[""]</code> (только пустая строка) • Строка из максимального символа алфавита → следующая строка длиннее
Эффективность	• Ленивая генерация: не хранить все строки в памяти сразу • Использовать <code>concatMap</code> для объединения строк разной длины

Вариант 9. Числа-палиндромы

Компонент	Описание
Задача	Бесконечные потоки палиндромных чисел в разных системах счисления
<code>palindromesBase10 :: [Integer]</code>	Все натуральные числа, являющиеся палиндромами в десятичной системе
<code>isPalindromeBase10 :: Integer -> Bool</code>	Проверка, является ли число палиндромом в десятичной системе
<code>isPalindromeBase2 :: Integer -> Bool</code>	Проверка, является ли число палиндромом в двоичной системе
<code>dualPalindromes :: [Integer]</code>	Числа, являющиеся палиндромами одновременно в десятичной и двоичной системах
Граничные случаи	• <code>0</code> считается палиндромом в любой системе счисления
Эффективность	• Генерация через <code>filter</code>

	isPalindromeBase10 [0..] • Для двоичного представления использовать showIntAtBase или ручную конвертацию
--	---

Вариант 10. Бесконечный поток подмножеств

Компонент	Описание
Задача	Генерация всех подмножеств конечного списка в порядке возрастания размера
<code>subsetsStream :: [a] -> [[a]]</code>	Все подмножества списка, упорядоченные сначала по размеру, затем лексикографически
<code>subsetsOfSize :: Int -> [a] -> [[a]]</code>	Все подмножества заданного размера
<code>combinations :: Int -> [a] -> [[a]]</code>	Комбинаторная функция: выбор <code>k</code> элементов из списка
Граничные случаи	• Пустой список $\rightarrow []$ (одно подмножество — пустое) • <code>subsetsOfSize 0 xs</code> $\rightarrow []$ • <code>subsetsOfSize k xs</code> при <code>k > length xs</code> $\rightarrow []$
Эффективность	• Ленивая генерация: <code>concatMap (\k -> subsetsOfSize k xs) [0..length xs]</code> • Рекурсивная реализация <code>combinations</code> через выбор первого элемента

Вариант 11. Последовательность Коллатца

Компонент	Описание
Задача	Генерация последовательностей Коллатца для всех натуральных чисел
<code>collatzStep :: Integer -> Integer</code>	Один шаг последовательности Коллатца: • Если чётное: <code>n / 2</code> • Если нечётное: <code>3n + 1</code>
<code>collatzSeq :: Integer -> [Integer]</code>	Полная последовательность Коллатца от начального числа до 1
<code>collatzLength :: Integer -> Int</code>	Бесконечный список длин последовательностей для всех натуральных чисел

<code>longestCollatzUnder :: Integer -> (Integer, Int)</code>	Число $< n$ с самой длинной последовательностью и длина этой последовательности
Граничные случаи	<code>collatzSeq 1</code> $\rightarrow [1]$ <code>collatzLength 1</code> $\rightarrow 0$ (ноль шагов)
Эффективность	- Мемоизация через бесконечный список: <code>collatzLengths = map collatzLength [1..]</code> - Для больших чисел использовать <code>Integer</code> вместо <code>Int</code>

Вариант 12. Поток всех пифагоровых троек

Компонент	Описание
Задача	Генерация всех примитивных пифагоровых троек <code>(a,b,c)</code> без дубликатов
<code>pythagoreanTriples :: [(Integer, Integer, Integer)]</code>	Бесконечный поток всех примитивных пифагоровых троек, упорядоченных по возрастанию <code>c</code>
<code>isPythagorean :: (Integer, Integer, Integer) -> Bool</code>	Проверка условия $a^2 + b^2 = c^2$
<code>isPrimitive :: (Integer, Integer, Integer) -> Bool</code>	Проверка, что тройка примитивная (наибольший общий делитель = 1)
<code>triplesWithCUnder :: Integer -> [(Integer, Integer, Integer)]</code>	Все тройки с <code>c < n</code>
Граничные случаи	- Гарантируется <code>a < b < c</code> - Исключаются кратные тройки (только примитивные)
Эффективность	- Генерация по формуле Евклида: <code>a = m²-n², b = 2mn, c = m²+n²</code> для <code>m > n</code> - Фильтрация через <code>filter isPrimitive</code>

Вариант 13. Мемоизация функции Аккермана

Компонент	Описание
Задача	Реализация функции Аккермана с мемоизацией через бесконечную таблицу
<code>ackermann :: Integer -> Integer -> Integer</code>	Функция Аккермана: $A(0, n) = n + 1$ $A(m, 0) = A(m - 1, 1)$ $A(m, n) = A(m - 1, A(m, n - 1))$

<code>ackTable :: [[Integer]]</code>	Бесконечная таблица значений Аккермана: <code>ackTable !! m !! n = A(m,n)</code>
<code>ackMemoized :: Integer -> Integer -> Integer</code>	Мемоизированная версия через <code>ackTable</code>
Граничные случаи	- Отрицательные аргументы — не определены <code>ackermann 4 1</code> уже очень большое число (может не поместиться в памяти)
Эффективность	- Без мемоизации <code>ackermann 3 3</code> вычисляется очень долго (экспоненциальная сложность) - С мемоизацией через <code>ackTable = [[ack m n]]</code>

Вариант 14. Бесконечный список чисел без цифры 9

Компонент	Описание
Задача	Генерация всех натуральных чисел, не содержащих цифру 9 в десятичном представлении
<code>noNineNumbers :: [Integer]</code>	Бесконечный список чисел, в десятичной записи которых нет цифры 9
<code>containsNine :: Integer -> Bool</code>	Проверка, содержит ли число цифру 9
<code>countNoNineUnder :: Integer -> Int</code>	Количество чисел без цифры 9 в диапазоне <code>[0, n)</code>
<code>nthNoNine :: Int -> Integer</code>	<code>n</code> -е число без цифры 9 (нумерация с 0)
Граничные случаи	<code>0</code> не содержит цифру 9 → включено <code>9, 19, 29, ..., 90-99, 109, ...</code> исключены
Эффективность	- Генерация через <code>filter (not . containsNine) [0..]</code> - Интересный факт: асимптотическая плотность таких чисел стремится к 0

Вариант 15. Поток приближений числа π

Компонент	Описание
Задача	Вычисление числа π через ряд Лейбница с ленивой генерацией приближений
<code>piApproximations :: [Double]</code>	Бесконечный поток последовательных

	приближений π по формуле Лейбница: $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$
<code>leibnizTerm :: Int -> Double</code>	<code>n</code> -й член ряда Лейбница (с учётом знака)
<code>termsNeededForPrecision :: Double -> Int</code>	Количество членов ряда, необходимых для достижения заданной точности
<code>piWithPrecision :: Double -> Double</code>	Вычисление π с заданной абсолютной точностью
Граничные случаи	• Точность ≤ 0 — не определена • Очень высокая точность (1e-10) требует миллионы членов ряда
Эффективность	• Ленивая генерация через <code>scanl (+) 0 (map leibnizTerm [0..])</code> • Для ускорения можно использовать ряд Мачина или другие формулы